

Database Systems Lab Architecture

Christian Rauch

(changed: August 24, 2025)

Overview

Frontend HTML, CSS, JavaScript



HTTP, AJAX, REST

Backend Python, Flask, WSGI



DB-API, mysql-connector-python

Database MariaDB, SQL

Frontend / HTML

The head section contains meta data, scripts, and styles.

```
1 <!doctype html>
2 <html>
3   <head>
4     <title>...</title>
5     <meta charset="UTF-8">
6     <link rel="stylesheet" href="styles.css">
7     <script src="common.js"></script>
8   </head>
```

The body section contains the rendered and interactive elements.

```
1   <body>
2     ...
3   </body>
4 </html>
```

Use the browser debug function (F12) to inspect the DOM, scripts, styles, network traffic, and more.

Backend / HTML Request Handling

In Flask, you map routes (URL patterns) to functions.

```
1 from flask import Flask, render_template
2 app = Flask(__name__)
3
4 @app.route('/profile/<uid>')
5 def profile(uid): # load uid
6     u = {"name": "Penny Low", "id": "c03929"}
7     return render_template('profile.html', user=u)
```

The HTML template ('profile.html') will be processed by Jinja2:

```
1 <!doctype html>
2 <html>
3     <head>
4         <title>Profile: {{ user.id }}</title>
5     </head>
6     <body>
7         <h1>Hello {{ user.name }}</h1>
8     </body>
9 </html>
```

Frontend / AJAX

AJAX is used for asynchronous communication with the server without full page reloads.

```
1 fetch('/api/data')
2   .then(response => {
3     if (!response.ok) {
4       throw new Error('...');
5     }
6     return response.json();
7   })
8   .then(data => {
9     ... // process the loaded data
10        // e.g., data.forEach(d => { ... })
11        // e.g., console.log(data)
12   })
13   .catch(e => console.error('...', e));
```

Backend / AJAX Request Handling

Flask handles requests and responses on the server side:

```
1 from flask import Flask, jsonify
2
3 app = Flask(__name__)
4
5 @app.route('/api/data', methods=['GET'])
6 def get_data():
7     data = [
8         {"name": "Mary Doe", "id": "d00221"},
9         {"name": "Jill Woe", "id": "z09431"}
10    ]
11    return jsonify(data)
12
13 if __name__ == '__main__':
14     app.run(debug=True)
```

Backend / Virtual Environment

Virtual environments isolate projects and allow to manage dependencies separately. This ensures predictable workflows and reproducible deployments.

Install venv using pip, create a virtual environment, and activate it.

```
1 $ python -m venv .venv
2 $ source .venv/bin/activate
```

Get a list of currently installed components.

```
1 $ python -m pip freeze
```

Install a list of components.

```
1 $ python -m pip install -r ./requirements.txt
```

Backend / Dependencies

File requirements.txt lists all direct dependencies (i.e., libraries) required by your system. You usually specify the exact versions.

```
1 flask==3.0.3
2 uwsgi==2.0.26
3 mysql-connector-python==9.0.0
4 ...
```

Do not list indirect sub-dependencies (i.e., dependencies of your dependencies), this is redundant.

Environments are not portable, create them in place. The directory should be added to .gitignore.

```
1 **/__pycache__
2 .venv
```


Conclusion

Thank you for your attention!